

RentRepair Log

Required Document - Assessment Task 3

Section 1: Problem Statement

The problem is that NSW private renters often need to manage repairs across scattered evidence: messages, emails, photos, files, dates, and verbal follow-up notes. When a repair is delayed, the renter needs a clear written timeline that shows what was reported, when it was reported, what evidence exists, and what follow-up action is due. Without that timeline, escalation becomes slower and more stressful because the renter must reconstruct events from several apps.

The primary stakeholder is a NSW private renter managing repair communication with a landlord or property manager. The context is ordinary residential tenancy, not commercial property management. The human cost is practical and personal: lost time, missed follow-ups, uncertainty about urgent repairs, and weaker evidence if the dispute reaches Fair Trading or the Tribunal.

Real-world evidence: NSW Government renter guidance describes how tenants get repairs done, distinguishes urgent repairs, and describes dispute pathways. NSW Government dispute guidance also advises tenants to keep records and evidence where disputes may arise. Sources: <https://www.nsw.gov.au/housing-and-construction/renting-a-place-to-live/getting-repairs-done>, <https://www.nsw.gov.au/housing-and-construction/rules/urgent-repairs-residential-rental-properties>, and <https://www.nsw.gov.au/housing-and-construction/renting-a-place-to-live/resolving-residential-tenancy-disputes>.

Section 2: Design Justification

An iOS app is appropriate because the renter records evidence at the moment it appears. Photos, shared messages, PDFs, and links are already on the phone. Offline access matters because a repair timeline should remain available during inspections, calls, or unreliable connectivity. A website could store the same data, but it would not integrate as directly with the Share sheet or Home Screen widget.

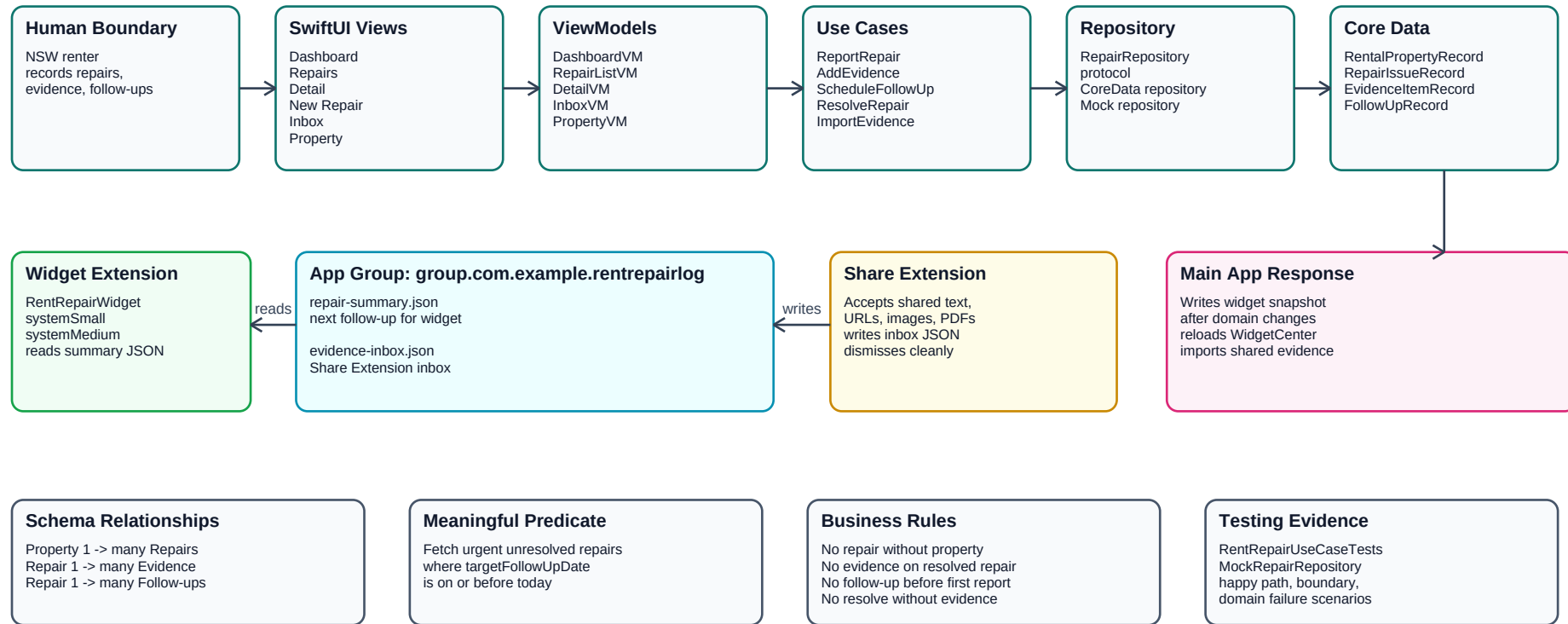
The widget exists for a specific scenario: the renter needs to see the next repair follow-up without opening the app. It reads a small App Group snapshot and supports systemSmall and systemMedium families. The Share Extension exists for another scenario: evidence starts in Messages, Mail, Photos, Files, or Safari. It writes evidence inbox items to the App Group so the app can attach them to the correct repair.

Core Data was selected because this data is private, single-user, structured, and useful offline. The schema has one rental property with many repair issues, and each repair issue has many evidence items and follow-up notes. The repository query for urgent unresolved repairs due for follow-up reflects the domain need: the renter needs to know what requires attention now.

Entity	Purpose	Relationships	Business Rule / Use
RentalPropertyRecord	Address and agent contact	Has many repairs	A repair cannot be reported until a property exists
RepairIssueRecord	Repair title, urgency, status, and follow-up date	Belongs to property; has evidence and follow-ups	Urgent unresolved due repairs appear on dashboard and widget
EvidenceItemRecord	Evidence imported or entered by renter	Belongs to one repair	Cannot be added to a resolved repair
FollowUpRecord	Follow-up due date and note	Belongs to one repair	Cannot be before the first report date

Architecture Diagram: RentRepair Log

Primary data flow: renter reports a repair -> use case applies rules -> repository persists -> UI and widget refresh.



Section 4: Reflective Report

Word count: 725

I chose rental repairs because it is a small, concrete problem where the technical requirements do not feel artificial. NSW renter guidance already expects repair issues to be communicated clearly, escalated through Fair Trading or the Tribunal if unresolved, and supported by records and evidence. That made the stakeholder precise: a NSW private renter trying to manage one home, one agent or landlord, and a timeline of defects. I did not need to invent a dramatic scenario. The everyday friction is enough: photos sit in Photos, messages sit in Messages, emails sit in Mail, and the follow-up date often lives only in the renter's memory.

My understanding changed while building the app. At first the obvious feature seemed to be a generic repair list. The more important feature became chronology. A renter does not just need to know that a window is broken; they need to show when it was first reported, what evidence existed, when they followed up, and whether the issue remained urgent. That is why the detail screen is evidence-heavy and why resolving a repair requires at least one evidence item. This rule is slightly strict, but it protects the core value of the system: a defensible timeline.

The WidgetKit extension serves the moment when the renter is busy and does not want to open the app just to check what needs attention. Removing the widget would not break the database, but it would make the app easier to forget, which is exactly the failure pattern the solution is meant to reduce. The widget reads only a summary from the App Group rather than opening Core Data directly, because extensions should stay small and because the main app already owns the domain rules.

The Share Extension is more important than the widget. Repair evidence usually starts outside the app: a text from the agent, an email, a PDF quote, a Safari link, or a photo. If the renter must manually copy everything later, the evidence trail becomes unreliable. The extension writes shared evidence metadata into an App Group inbox, and the main app asks the renter which repair it belongs to. I deliberately did not let the extension attach evidence directly to a repair, because the extension does not have enough context to choose safely.

I chose Core Data instead of CloudKit because the data is private, single-user, and useful offline. A repair dispute timeline may include sensitive home details, contact names, and evidence notes. Sync would be convenient, but it is not required for the stakeholder workflow and would add account, merge, and privacy complexity. The key schema decision was making `RepairIssueRecord` the centre of the model, with evidence and follow-ups as child records. That matches the renter's mental model: everything exists because it supports a particular repair issue.

The main architecture pressure came from extensions. The cleanest domain architecture keeps all data behind use cases and repositories, but WidgetKit and Share Extensions cannot simply use the app's ViewModels. I resolved this by using the App Group as a boundary. The main app writes a widget snapshot after relevant data changes, and the Share Extension writes inbox items that the app later imports through a use case. This keeps extension code simple while preserving the app's layered architecture.

The other pressure was between speed and proof. A renter wants to capture evidence quickly, especially when sharing from another app, but the assessment also requires meaningful domain rules. I chose to let the Share Extension capture evidence into an inbox first, then require the main app to attach it to a specific repair. That adds one review step, but it prevents evidence being filed under the wrong issue and keeps the timeline trustworthy.

AI assistance was used throughout the build for planning, code generation, documentation drafting, and review. The useful parts were rapid scaffolding and cross-checking the rubric against the implementation. The parts that needed careful evaluation were Xcode project target wiring, extension Info.plist behavior, and date-sensitive unit test data. I verified the output by building the app, running the unit tests, checking target membership, and comparing the README, report, and code against the canonical specification. I

would not submit the AI output unreviewed, because small inconsistencies in bundle identifiers, App Group names, or entity names would be easy to miss and costly in marking.

References

NSW Government. Getting repairs done on a rental property. <https://www.nsw.gov.au/housing-and-construction/renting-a-place-to-live/getting-repairs-done>

NSW Government. Urgent repairs in residential rental properties. <https://www.nsw.gov.au/housing-and-construction/rules/urgent-repairs-residential-rental-properties>

NSW Government. Resolving residential tenancy disputes. <https://www.nsw.gov.au/housing-and-construction/renting-a-place-to-live/resolving-residential-tenancy-disputes>